

SECTION 1: Registration and Login Features

referencing: Baeldung regex tutorial - <https://www.baeldung.com/java-regex-validate-phone-numbers>

```
//LOGIN CLASS//
static class Login {
    private String firstName;
    private String lastName;
    private String username;
    private String password;
    private String cellPhoneNumber;

    public Login(String firstName, String lastName, String username, String password, String
cellPhoneNumber) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.username = username;
        this.password = password;
        this.cellPhoneNumber = cellPhoneNumber;
    }

    public boolean checkUserName() {
        return username.contains("_") && username.length() <= 5;
    }

    public boolean checkPasswordComplexity() {
        if (password.length() < 8) return false;
        boolean hasCapital = false, hasNumber = false, hasSpecial = false;
        for (char c : password.toCharArray()) {
            if (Character.isUpperCase(c)) hasCapital = true;
            else if (Character.isDigit(c)) hasNumber = true;
            else if (!Character.isLetterOrDigit(c)) hasSpecial = true;
        }
        return hasCapital && hasNumber && hasSpecial;
    }

    public boolean checkCellPhoneNumber() {
        //REGEX SOURCED AND ADAPTED FROM://
        //https://www.baeldung.com/java-regex-validate-phone-numbers//
        return cellPhoneNumber.matches("^\\+27\\d{9}$");
    }

    public boolean loginUser(String enteredUsername, String enteredPassword) {
        return this.username.equals(enteredUsername) &&
this.password.equals(enteredPassword);
    }

    public String returnLoginStatus(String enteredUsername, String enteredPassword) {
```

```

        if (loginUser(enteredUsername, enteredPassword)) {
            return "Welcome " + firstName + " " + lastName + ", it is great to see you again!";
        } else {
            return "Username or password incorrect, please try again.";
        }
    }

    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
}

```

Welcome Menu and Registration/Login Flow (from main method)

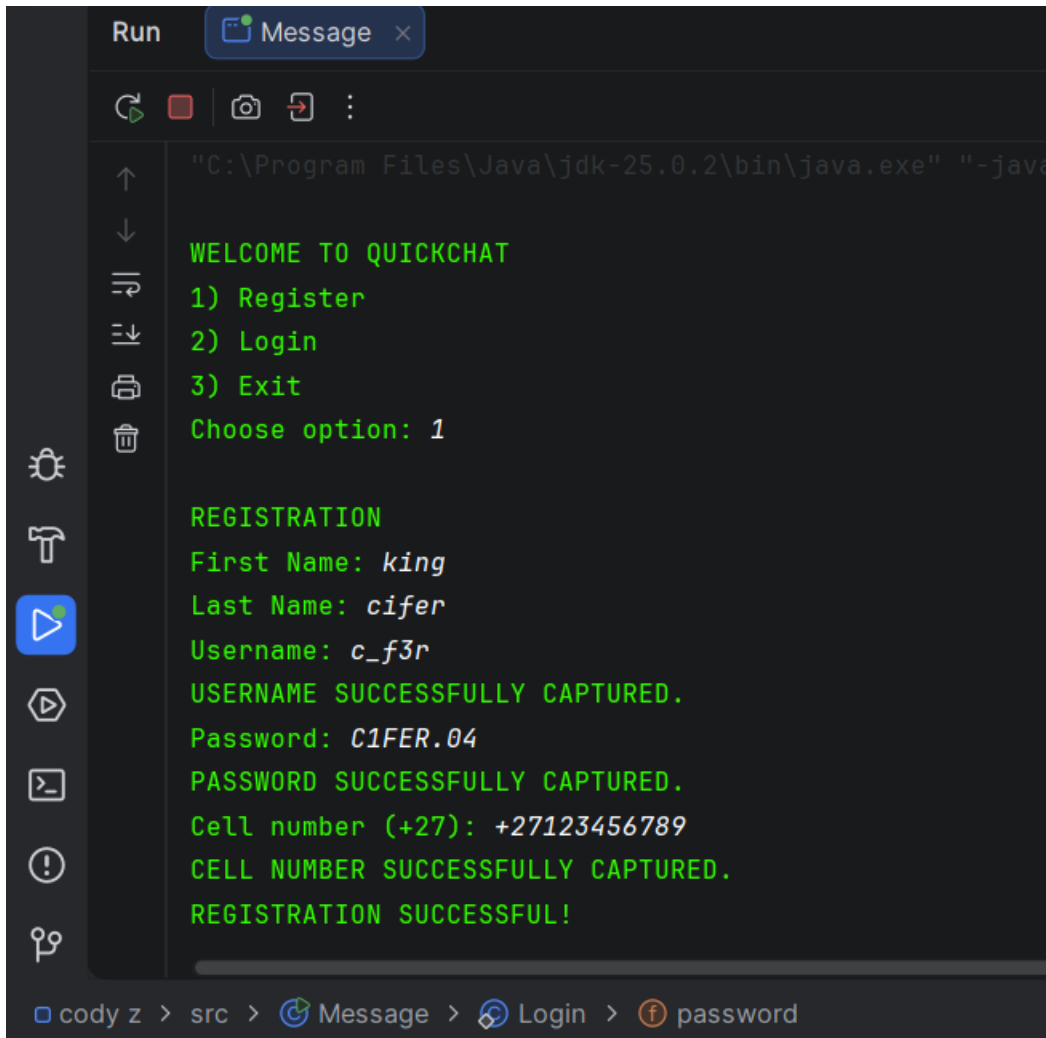
```

//WELCOME SCREEN TO REGISTER OR LOGIN//
boolean welcomed = true;
while (welcomed) {
    System.out.println("\n WELCOME TO QUICKCHAT ");
    System.out.println("1) Register");
    System.out.println("2) Login");
    System.out.println("3) Exit");
    System.out.print("Choose option: ");
    int welcome;
    try {
        welcome = Integer.parseInt(scanner.nextLine().trim());
    } catch (NumberFormatException e) {

```

```
        System.out.println("Invalid option.");
        continue;
    }
    switch (welcome) {
        case 1:
            // ... registration logic: validates username, password and cell
            // number using checkUserName(), checkPasswordComplexity() and
            // checkCellPhoneNumber() in while loops until each is valid.
            break;
        case 2:
            // ... login logic: calls returnLoginStatus() and exits the
            // welcome loop only if loginUser() returns true.
            break;
        case 3:
            System.out.println("Goodbye!");
            scanner.close();
            return;
        default:
            System.out.println("Invalid option.");
    }
}
```

SUCCESSFUL LOGIN AND REGISTRATION



The screenshot shows a Java application window titled "Message" with a dark theme. The application is running, as indicated by the "Run" button in the top-left corner. The main content area displays the following text in green:

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-java  
WELCOME TO QUICKCHAT  
1) Register  
2) Login  
3) Exit  
Choose option: 1  
  
REGISTRATION  
First Name: king  
Last Name: cifer  
Username: c_f3r  
USERNAME SUCCESSFULLY CAPTURED.  
Password: C1FER.04  
PASSWORD SUCCESSFULLY CAPTURED.  
Cell number (+27): +27123456789  
CELL NUMBER SUCCESSFULLY CAPTURED.  
REGISTRATION SUCCESSFUL!
```

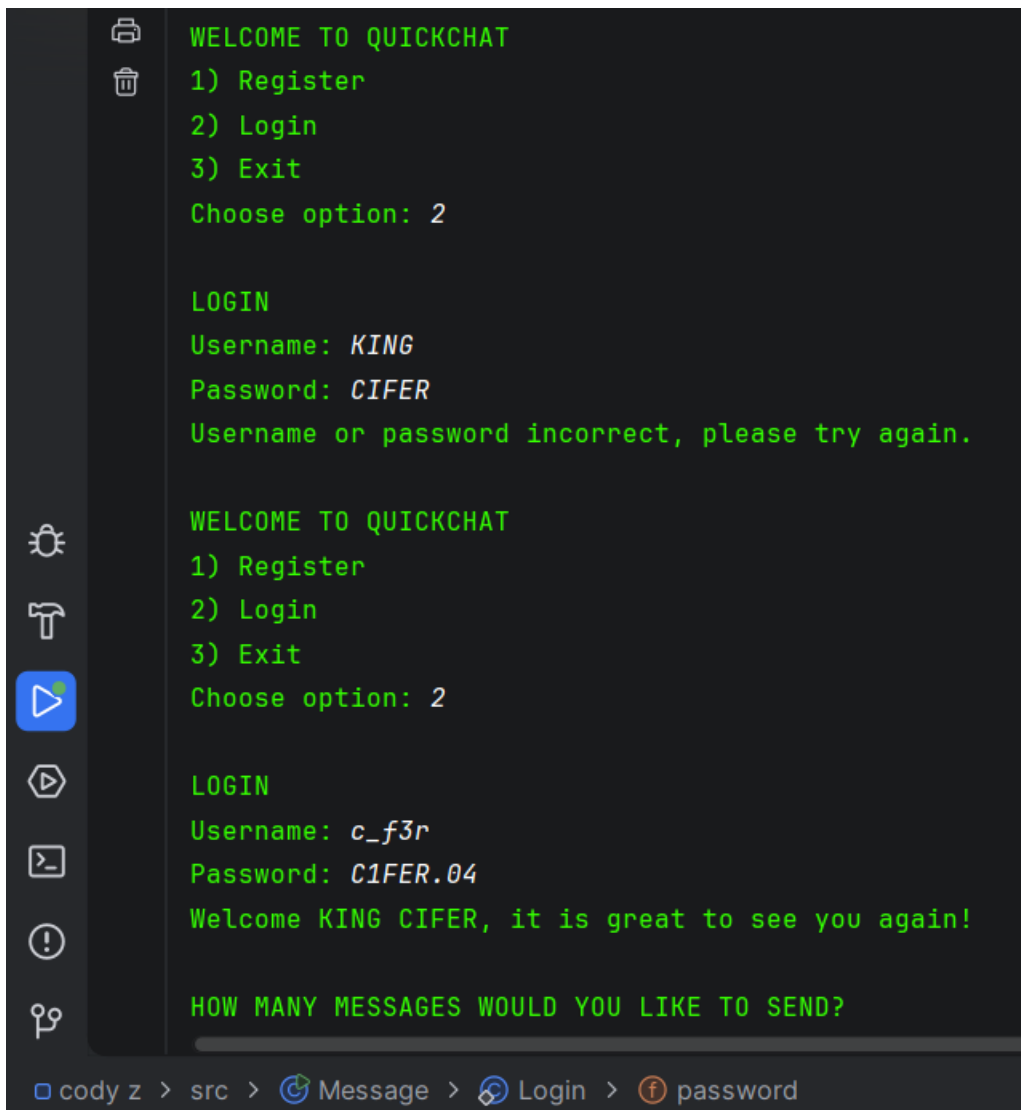
The left sidebar contains several icons for application functionality, including a play button (highlighted in blue), a gear, a hammer, a document, a magnifying glass, a warning sign, and a person icon. The bottom status bar shows the file path: `cody z > src > Message > Login > password`.

```
WELCOME TO QUICKCHAT
1) Register
2) Login
3) Exit
Choose option: 1

REGISTRATION
First Name: KING
Last Name: CIFER
Username: c1fer
USERNAME INVALID (MUST CONTAIN '_' AND BE ≤ 5 CHARACTERS).
Username: c_f3r
USERNAME SUCCESSFULLY CAPTURED.
Password: c1fer
PASSWORD INVALID (8+ CHARS, CAPITAL, NUMBER, SPECIAL CHAR).
Password: C1FER.04
PASSWORD SUCCESSFULLY CAPTURED.
Cell number (+27): 0712345678
INVALID CELL NUMBER FORMAT (+27 REQUIRED).
Cell number (+27): +27123456789
CELL NUMBER SUCCESSFULLY CAPTURED.
REGISTRATION SUCCESSFUL!

WELCOME TO QUICKCHAT
1) Register
2) Login
3) Exit
Choose option: |
```

cody z > src > Message > Login > password



```
WELCOME TO QUICKCHAT
1) Register
2) Login
3) Exit
Choose option: 2

LOGIN
Username: KING
Password: C1FER
Username or password incorrect, please try again.

WELCOME TO QUICKCHAT
1) Register
2) Login
3) Exit
Choose option: 2

LOGIN
Username: c_f3r
Password: C1FER.04
Welcome KING C1FER, it is great to see you again!

HOW MANY MESSAGES WOULD YOU LIKE TO SEND?
```

cody z > src > Message > Login > password

SECTION 2 – SENDING MESSAGES

MessageData Class - Core Methods

```
//MESSAGE CLASS//
static class MessageData {
    private String messageId;
    private int messageNumber;
    private String recipient;
    private String messageText;
    private String messageHash;
    private String flag;
    static int totalSent = 0;
    static ArrayList<MessageData> sentMessages = new ArrayList<>();
    static ArrayList<MessageData> storedMessages = new ArrayList<>();
    static ArrayList<MessageData> disregardMessages = new ArrayList<>();
    static ArrayList<String> messageHashes = new ArrayList<>();
    static ArrayList<String> messageIDs = new ArrayList<>();
    public MessageData(int messageNumber, String recipient, String messageText) {
        this.messageNumber = messageNumber;
        this.recipient = recipient;
        this.messageText = messageText;
        this.messageID = generateMessageID();
        this.messageHash = createMessageHash();
    }
    //CONSTRUCTOR WITH FIXED ID, USED FOR PRELOADED TEST DATA SO MESSAGE IDS ARE
    REPEATABLE//
    public MessageData(int messageNumber, String recipient, String messageText, String
    fixedID) {
        this.messageNumber = messageNumber;
        this.recipient = recipient;
        this.messageText = messageText;
        this.messageID = fixedID;
        this.messageHash = createMessageHash();
    }
    private String generateMessageID() {
        Random rand = new Random();
        long id = (long)(rand.nextDouble() * 9_000_000_000L) + 1_000_000_000L;
        return String.valueOf(id);
    }
    public boolean checkMessageID() {
```

```

        return messageID.length() <= 10;
    }
    public String checkRecipientCell() {
        if (recipient.startsWith("+") && recipient.length() <= 13) {
            return "Cell phone number successfully captured.";
        } else {
            return "Cell phone number is incorrectly formatted or does not contain an international
code. Pl
correct the number and try again.";
        }
    }
    public String createMessageHash() {
        String[] words = messageText.trim().split("\\s+");
        String firstWord = words[0];
        String lastWord = words[words.length - 1];
        String idPrefix = messageID.substring(0, Math.min(2, messageID.length()));
        return (idPrefix + ":" + messageNumber + ":" + firstWord + lastWord).toUpperCase();
    }
    public String checkMessageLength() {
        if (messageText.length() <= 250) {
            return "Message ready to send.";
        } else {

```



```

        int over = messageText.length() - 250;
        return "Message exceeds 250 characters by " + over + "; please reduce the size.";
    }
}

public String sendMessage(int choice) {
    switch (choice) {
        case 1:
            flag = "Sent";
            totalSent++;
            sentMessages.add(this);
            messageHashes.add(messageHash);
            messageIDs.add(messageID);
            return "Message successfully sent.";
        case 2:
            flag = "Disregarded";
            disregardMessages.add(this);
            return "Press 0 to delete the message.";
        case 3:
            flag = "Stored";
            storedMessages.add(this);
            messageHashes.add(messageHash);
            messageIDs.add(messageID);
            return "Message successfully stored.";
        default:
            return "Invalid option.";
    }
}

public static String printMessages() {
    if (sentMessages.isEmpty()) return "No messages sent.";
    StringBuilder sb = new StringBuilder();
    for (MessageData m : sentMessages) {
        sb.append("Message ID: ").append(m.messageID)
          .append(" | Hash: ").append(m.messageHash)
          .append(" | Recipient: ").append(m.recipient)
          .append(" | Message: ").append(m.messageText)
          .append("\n");
    }
    return sb.toString();
}

public static int returnTotalMessages() {
    return totalSent;
}

```

Main Method - Welcome Message, Menu Loops and Send Logic

```
//MAIN//
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    Login user = null;
    //WELCOME SCREEN TO REGISTER OR LOGIN//
    boolean welcomed = true;
    while (welcomed) {
        System.out.println("\n WELCOME TO QUICKCHAT ");
        System.out.println("1) Register");
        System.out.println("2) Login");
        System.out.println("3) Exit");
        System.out.print("Choose option: ");
        int welcome;
        try {
            welcome = Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid option.");
            continue;
        }
        switch (welcome) {
            case 1:
                System.out.println("\nREGISTRATION ");
                System.out.print("First Name: ");
                String regFirst = scanner.nextLine();
                System.out.print("Last Name: ");
                String regLast = scanner.nextLine();
```

```

String regUsername;
while (true) {
    System.out.print("Username: ");
    regUsername = scanner.nextLine();
    Login tempU = new Login(regFirst, regLast, regUsername, "", "+27000000000");
    if (tempU.checkUserName()) {
        System.out.println("Username successfully captured.");
        break;
    } else {
        System.out.println("Username is not correctly formatted; please ensure that
your use
contains an underscore and is no more than five characters in length.");
    }
}
String regPassword;
while (true) {
    System.out.print("Password: ");
    regPassword = scanner.nextLine();
    Login tempP = new Login(regFirst, regLast, regUsername, regPassword,
"+27000000000");
    if (tempP.checkPasswordComplexity()) {
        System.out.println("Password successfully captured.");
        break;
    } else {
        System.out.println("Password is not correctly formatted; please ensure that the
pass
contains at least 8 characters, a capital letter, a number and a special character.");
    }
}
String regCell;
while (true) {
    System.out.print("Cell number (+27): ");
    regCell = scanner.nextLine();
    Login tempC = new Login(regFirst, regLast, regUsername, regPassword, regCell);
    if (tempC.checkCellPhoneNumber()) {
        System.out.println("Cellphone number successfully captured.");
        break;
    } else {
        System.out.println("Cell phone number incorrectly formatted or does not
contain inte
code ( +27 ).");
    }
}

```

```

    }
    user = new Login(regFirst, regLast, regUsername, regPassword, regCell);
    System.out.println("Registration successful! You can now log in.");
    break;
case 2:
    if (user == null) {
        System.out.println("No user registered. Please register first.");
        break;
    }
    System.out.println("\nLOGIN ");
    System.out.print("Username: ");
    String enteredUsername = scanner.nextLine();
    System.out.print("Password: ");
    String enteredPassword = scanner.nextLine();
    System.out.println(user.returnLoginStatus(enteredUsername, enteredPassword));
    if (user.loginUser(enteredUsername, enteredPassword)) {
        welcomed = false;
    }
    break;
case 3:
    System.out.println("Goodbye!");
    scanner.close();
    return;
default:
    System.out.println("Invalid option.");
}
}

//PRELOAD THE PART 3 TEST DATA SO THE STORED MESSAGES MENU HAS DATA TO WORK
WITH//
MessageData.preloadTestData();
//ASK HOW MANY MESSAGES TO SEND//
System.out.print("\nHow many messages would you like to send? ");
int numMessages = 0;
try {
    numMessages = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid number. Defaulting to 0.");
}

```

```

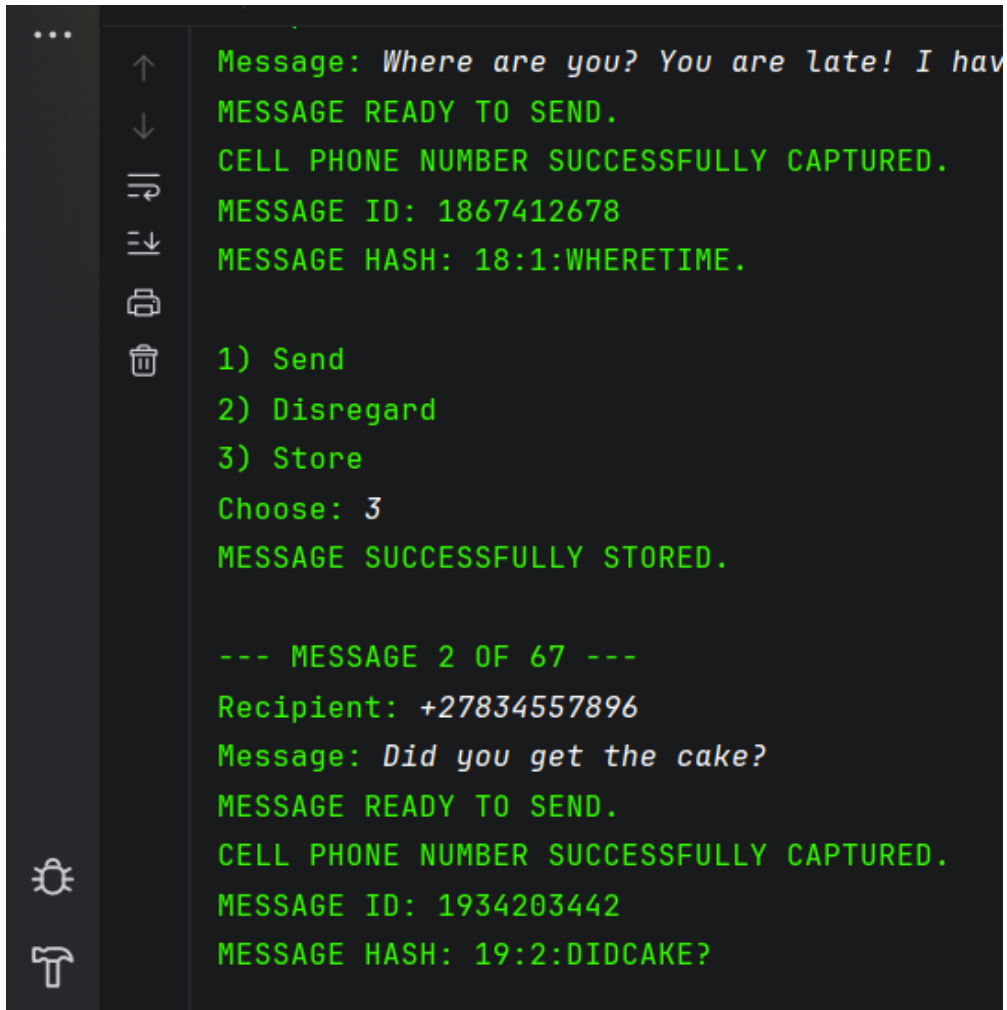
}
//MAIN MENU LOOP//
boolean running = true;
while (running) {
    System.out.println("\n WELCOME TO QUICKCHAT");
    System.out.println("1) Send Messages");
    System.out.println("2) Show Recently Sent Messages");
    System.out.println("3) Stored Messages");
    System.out.println("4) Quit");
    System.out.print("Choose option: ");
    int choice;
    try {
        choice = Integer.parseInt(scanner.nextLine().trim());
    } catch (NumberFormatException e) {
        System.out.println("Invalid option.");
        continue;
    }
    switch (choice) {
        case 1:
            //FOR LOOP - SENDS THE NUMBER OF MESSAGES SPECIFIED BY THE USER//
            for (int i = 1; i <= numMessages; i++) {
                System.out.println("\n--- Message " + i + " of " + numMessages + " ---");
                System.out.print("Recipient cell (+27...): ");
                String recipient = scanner.nextLine().trim();
                String messageText;
                while (true) {
                    System.out.print("Message: ");
                    messageText = scanner.nextLine();
                    if (messageText.length() <= 250) {
                        System.out.println("Message ready to send.");
                        break;
                    } else {
                        int over = messageText.length() - 250;
                        System.out.println("Message exceeds 250 characters by " + over
                            + "; please reduce the size.");
                    }
                }
                MessageData msg = new MessageData(i, recipient, messageText);
                System.out.println(msg.checkRecipientCell());
                System.out.println("Message ID generated: " + msg.getMessageID());
                System.out.println("Message Hash: " + msg.getMessageHash());
                System.out.println("\n1) Send Message");
            }
        }
    }
}

```

```

        System.out.println("2) Disregard Message");
        System.out.println("3) Store Message to send later");
        System.out.print("Choose: ");
        int sendChoice;
        try {
            sendChoice = Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            sendChoice = 2;
        }
        System.out.println(msg.sendMessage(sendChoice));
        System.out.println("\n Message Details ");
        System.out.println("Message ID  : " + msg.getMessageID());
        System.out.println("Message Hash : " + msg.getMessageHash());
        System.out.println("Recipient   : " + msg.getRecipient());
        System.out.println("Message    : " + msg.getMessageText());
    }
    System.out.println("\nTotal messages sent: " + MessageData.returnTotalMessages());
    //SAVE STORED MESSAGES TO JSON AFTER EACH SEND SESSION//
    if (!MessageData.storedMessages.isEmpty()) {
        MessageData.storeMessage("stored_messages.json");
    }
    break;
case 2:
    System.out.println(MessageData.printMessages());
    break;

```



```

--- MESSAGE 4 OF 67 ---
Recipient: 0796185198
Message: HELLO
MESSAGE READY TO SEND.
CELL PHONE NUMBER IS INCORRECTLY FORMATTED OR DOES NOT CONTAIN AN INTERNATIONAL CODE. PLEASE CORRECT THE NUMBER AND TRY AGAIN.
MESSAGE ID: 9449032583
MESSAGE HASH: 94:4:HELLOHELLO

```

SECTION 3- JSON Storage

storeMessage() Method

```

//STORES STORED MESSAGES TO A JSON FILE//
//FILE WRITING APPROACH ADAPTED FROM://
//https://www.baeldung.com/java-write-to-file//
public static void storeMessage(String filename) {
    StringBuilder json = new StringBuilder();
    json.append("[\n");
    for (int i = 0; i < storedMessages.size(); i++) {
        MessageData m = storedMessages.get(i);
        json.append(" {\n");
        json.append("  \"messageID\": \"\").append(m.messageID).append("\",\n");
        json.append("  \"messageHash\": \"\").append(m.messageHash).append("\",\n");
        json.append("  \"recipient\": \"\").append(m.recipient).append("\",\n");
        json.append("  \"message\": \"\").append(m.messageText).append("\",\n");
        json.append("  \"flag\": \"\").append(m.flag).append("\",\n");
        json.append(" }");
        if (i < storedMessages.size() - 1) json.append(",");
        json.append("\n");
    }
    json.append("]");
    try (FileWriter fw = new FileWriter(filename)) {
        fw.write(json.toString());
        System.out.println("Messages successfully stored to " + filename);
    } catch (IOException e) {
        System.out.println("Error writing JSON file: " + e.getMessage());
    }
}

```

preloadTestData() - Populates Arrays with Part 3 Test Data

```

//PRELOADS THE FIVE PART 3 TEST MESSAGES INTO THE ARRAYS BY CALLING

```



```

SENTMESSAGE//
//THIS POPULATES SENTMESSAGES, STOREDMESSAGES, DISREGARDMESSAGES,
MESSAGEHASHES AND MESSAGEIDS//
public static void preloadTestData() {
    MessageData m1 = new MessageData(1, "+27834557896", "Did you get the cake?",
"1000000001");
    MessageData m2 = new MessageData(2, "+27838884567", "Where are you? You are late!
I have asked you
to be on time.", "1000000002");
    MessageData m3 = new MessageData(3, "+27834484567", "Yohoooo, I am at your gate.",
"1000000003");
    MessageData m4 = new MessageData(4, "0838884567", "It is dinner time !",
"1000000004");
    MessageData m5 = new MessageData(5, "+27838884567", "Ok, I am leaving without
you.", "1000000005");
    m1.sendMessage(1);
    m2.sendMessage(3);
    m3.sendMessage(2);
    m4.sendMessage(1);
    m5.sendMessage(3);
}

```

PART 3 – ARRAYLIST TO STORE, SENT, DISREGARD, MESSAGE HASHES AND IDS

```

\      case 3:
        //STORED MESSAGES SUB-MENU//
        boolean storedMenu = true;
        while (storedMenu) {
            System.out.println("\nSTORED MESSAGES MENU");
            System.out.println("a) Display sender and recipient of all stored messages");
            System.out.println("b) Display the longest message");
            System.out.println("c) Search for a message by ID");
            System.out.println("d) Search for messages by recipient");
            System.out.println("e) Delete a message by hash");
            System.out.println("f) Display full report of sent messages");
            System.out.println("x) Back to main menu");
            System.out.print("Choose: ");
            String sub = scanner.nextLine().trim().toLowerCase();
            switch (sub) {
                case "a":

```

```

        System.out.println(MessageData.displaySenderAndRecipient());
        break;
    case "b":
        System.out.println("Longest message: " +
MessageData.displayLongestMessage());
        break;
    case "c":
        System.out.print("Enter Message ID: ");
        String searchID = scanner.nextLine().trim();
        System.out.println(MessageData.searchByMessageID(searchID));
        break;
    case "d":
        System.out.print("Enter Recipient number: ");
        String searchRec = scanner.nextLine().trim();
        System.out.println(MessageData.searchByRecipient(searchRec));
        break;
    case "e":
        System.out.print("Enter Message Hash: ");
        String hash = scanner.nextLine().trim();
        System.out.println(MessageData.deleteMessageByHash(hash));
        break;
    case "f":
        System.out.println(MessageData.displayReport());
        break;
    case "x":
        storedMenu = false;
        break;
    default:
        System.out.println("Invalid option.");
    }
}
break;
case 4:
    System.out.println("Goodbye!");
    running = false;
    break;
default:
    System.out.println("Invalid option.");
}
}
scanner.close();

```

}
}

Array and Reporting Methods (a-f)

//a) DISPLAYS THE MESSAGE NUMBER AND RECIPIENT FOR EVERY STORED MESSAGE//

```
public static String displaySenderAndRecipient() {
    if (storedMessages.isEmpty()) return "No stored messages.";
    StringBuilder sb = new StringBuilder();
    for (MessageData m : storedMessages) {
        sb.append("Message Number: ").append(m.messageNumber)
          .append(" | Recipient: ").append(m.recipient)
          .append("\n");
    }
    return sb.toString();
}
```

//b) FINDS AND RETURNS THE LONGEST MESSAGE OUT OF SENT, STORED AND DISREGARDED MESSAGES//

```
public static String displayLongestMessage() {
    ArrayList<MessageData> all = new ArrayList<>();
    all.addAll(sentMessages);
    all.addAll(storedMessages);
    all.addAll(disregardMessages);
    if (all.isEmpty()) return "No messages available.";
    MessageData longest = all.get(0);
    for (MessageData m : all) {
        if (m.messageText.length() > longest.messageText.length()) {
            longest = m;
        }
    }
    return longest.messageText;
}
```

//c) SEARCHES SENT AND STORED MESSAGES FOR A MATCHING MESSAGE ID AND RETURNS THE RECIPIENT AND

MESSAGE//

```
public static String searchByMessageID(String searchID) {
    ArrayList<MessageData> all = new ArrayList<>();
    all.addAll(sentMessages);
    all.addAll(storedMessages);
    for (MessageData m : all) {
        if (m.messageID.equals(searchID)) {
            return "Recipient: " + m.recipient + "\nMessage: " + m.messageText;
        }
    }
    return "Message ID not found.";
}
```

```

    }
    //d) SEARCHES SENT AND STORED MESSAGES AND RETURNS ALL MESSAGES FOR THE GIVEN
    RECIPIENT//
    public static String searchByRecipient(String searchRecipient) {
        ArrayList<MessageData> all = new ArrayList<>();
        all.addAll(sentMessages);
        all.addAll(storedMessages);
        StringBuilder sb = new StringBuilder();
        for (MessageData m : all) {
            if (m.recipient.equals(searchRecipient)) {
                sb.append(m.messageText).append("\n");
            }
        }
        if (sb.length() == 0) return "No messages found for that recipient.";
        return sb.toString();
    }
    //e) DELETES A MESSAGE FROM SENT OR STORED MESSAGES USING ITS MESSAGE HASH//
    public static String deleteMessageByHash(String hash) {
        ArrayList<MessageData> all = new ArrayList<>();
        all.addAll(sentMessages);
        all.addAll(storedMessages);
        for (MessageData m : all) {
            if (m.messageHash.equalsIgnoreCase(hash)) {
                String deletedText = m.messageText;
                sentMessages.remove(m);
                storedMessages.remove(m);
                messageHashes.remove(m.messageHash);
                messageIDs.remove(m.messageID);
                return "Message \"" + deletedText + "\" successfully deleted.";
            }
        }
        return "Hash not found. No message deleted.";
    }
    //f) BUILDS A FULL REPORT OF ALL SENT MESSAGES SHOWING MESSAGE HASH, RECIPIENT
    AND MESSAGE//

```

```
public static String displayReport() {  
    if (sentMessages.isEmpty()) return "No sent messages to report.";  
    StringBuilder sb = new StringBuilder();  
    for (MessageData m : sentMessages) {  
        sb.append("Message Hash: ").append(m.messageHash)  
          .append(" | Recipient: ").append(m.recipient)  
          .append(" | Message: ").append(m.messageText)  
          .append("\n");  
    }  
    return sb.toString();  
}  
  
public String getMessageID() { return messageID; }  
public String getRecipient() { return recipient; }  
public String getMessageText() { return messageText; }  
public String getMessageHash() { return messageHash; }  
public String getFlag() { return flag; }  
}
```

SECTION 5-Unit Testing

MessageTest.java - Full Code

```
package com.mycompany.message;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import static org.junit.jupiter.api.Assertions.*;
public class MessageTest {
    //RESET ALL STATIC STATE BEFORE EVERY TEST//
    @BeforeEach
    public void resetStatics() {
        Message.MessageData.totalSent = 0;
        Message.MessageData.sentMessages.clear();
        Message.MessageData.storedMessages.clear();
        Message.MessageData.disregardMessages.clear();
        Message.MessageData.messageHashes.clear();
        Message.MessageData.messageIDs.clear();
    }
    //LOGIN TESTS//
    @Test
    public void testUsernameValid() {
        //kyl_1 CONTAINS UNDERSCORE AND IS 5 CHARS - SHOULD PASS//
        Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
        "+27838968976");
        assertTrue(user.checkUserName());
    }
    @Test
    public void testUsernameInvalid() {
        //NO UNDERSCORE AND MORE THAN 5 CHARS - SHOULD FAIL//
        Message.Login user = new Message.Login("Kyle", "Jackson", "kyle!!!!", "Ch&&sec@ke99!",
        "+27838968976");
        assertFalse(user.checkUserName());
    }
    @Test
    public void testPasswordValid() {
        //Ch&&sec@ke99! HAS CAPITAL, NUMBER, SPECIAL - SHOULD PASS//
        Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
        "+27838968976");
        assertTrue(user.checkPasswordComplexity());
    }
}
```

```

@Test
public void testPasswordInvalid() {
    //password IS ALL LOWERCASE - SHOULD FAIL//
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "password",
    "+27838968976");
    assertFalse(user.checkPasswordComplexity());
}

@Test
public void testCellValid() {
    //+27838968976 MATCHES +27 FORMAT - SHOULD PASS//
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
    "+27838968976");
    assertTrue(user.checkCellPhoneNumber());
}

@Test
public void testCellInvalid() {
    //08966553 HAS NO INTERNATIONAL CODE - SHOULD FAIL//
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
    "08966553");
    assertFalse(user.checkCellPhoneNumber());
}

@Test
public void testLoginSuccess() {
    //CORRECT CREDENTIALS - SHOULD RETURN TRUE//
    Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
    "+27838968976");
    assertTrue(user.loginUser("kyl_1", "Ch&&sec@ke99!"));
}

@Test
public void testLoginFail() {

```



```

//WRONG PASSWORD - SHOULD RETURN FALSE//
Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
"+27838968976");
assertFalse(user.loginUser("kyl_1", "wrongpassword"));
}
@Test
public void testLoginStatusSuccess() {
//CORRECT CREDENTIALS - SHOULD RETURN WELCOME MESSAGE//
Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
"+27838968976");
assertEquals(
"Welcome Kyle Jackson, it is great to see you again!",
user.returnLoginStatus("kyl_1", "Ch&&sec@ke99!")
);
}
@Test
public void testLoginStatusFail() {
//WRONG CREDENTIALS - SHOULD RETURN FAILURE MESSAGE//
Message.Login user = new Message.Login("Kyle", "Jackson", "kyl_1", "Ch&&sec@ke99!",
"+27838968976");
assertEquals(
"Username or password incorrect, please try again.",
user.returnLoginStatus("kyl_1", "wrongpassword")
);
}
//MESSAGE LENGTH TESTS//
@Test
public void testMessageLengthValid() {
//SHORT MESSAGE - SHOULD RETURN READY TO SEND//
Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can
you join us
for dinner tonight?");
assertEquals("Message ready to send.", msg.checkMessageLength());
}
@Test
public void testMessageLengthTooLong() {
//260 CHARS IS 10 OVER LIMIT - SHOULD RETURN ERROR WITH OVERCOUNT//
String longMsg = "A".repeat(260);
Message.MessageData msg = new Message.MessageData(1, "+27718693002", longMsg);
assertTrue(msg.checkMessageLength().contains("exceeds 250 characters by 10"));
}
//RECIPIENT TESTS//

```

```

@Test
public void testRecipientValid() {
    //VALID +27 NUMBER - SHOULD RETURN SUCCESS//
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can
you join us
for dinner tonight?");
    assertEquals("Cell phone number successfully captured.", msg.checkRecipientCell());
}

@Test
public void testRecipientInvalid() {
    //NO INTERNATIONAL CODE - SHOULD RETURN FAILURE//
    Message.MessageData msg = new Message.MessageData(2, "08575975889", "Hi Keegan, did
you receive
the payment?");
    assertTrue(msg.checkRecipientCell().contains("incorrectly formatted or does not contain an
international code"));
}

//MESSAGE HASH TESTS//

@Test
public void testMessageHashUppercase() {
    //HASH MUST BE ALL UPPERCASE//
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can
you join us
for dinner tonight?");
    String hash = msg.getMessageHash();
    assertEquals(hash, hash.toUpperCase());
}

@Test
public void testMessageHashFormat() {
    //HASH MUST CONTAIN COLONS AND CORRECT WORD COMBINATION//
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can
you join us
for dinner tonight?");
    assertTrue(msg.getMessageHash().contains(":1:HITONIGHT?"));
}

//SENTMESSAGE OPTION TESTS//

@Test
public void testSentMessageSend() {
    //CHOICE 1 - SHOULD RETURN SENT MESSAGE//

```

```

    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can
you join us
for dinner tonight?");
    assertEquals("Message successfully sent.", msg.sendMessage(1));
}

@Test
public void testSendMessageDisregard() {
    //CHOICE 2 - SHOULD RETURN DISREGARD MESSAGE//
    Message.MessageData msg = new Message.MessageData(2, "08575975889", "Hi Keegan, did
you receive
the payment?");
    assertEquals("Press 0 to delete the message.", msg.sendMessage(2));
}

@Test
public void testSendMessageStore() {
    //CHOICE 3 - SHOULD RETURN STORED MESSAGE//
    Message.MessageData msg = new Message.MessageData(1, "+27718693002", "Hi Mike, can
you join us
for dinner tonight?");
    assertEquals("Message successfully stored.", msg.sendMessage(3));
}

//TOTAL MESSAGES TEST//

@Test
public void testReturnTotalMessages() {
    //SEND TWO MESSAGES - TOTAL SHOULD EQUAL 2//
    Message.MessageData m1 = new Message.MessageData(1, "+27718693002", "Hi Mike, can you
join us
for dinner tonight?");
    m1.sendMessage(1);
    Message.MessageData m2 = new Message.MessageData(2, "+27718693002", "Did you get the
cake?");
    m2.sendMessage(1);
    assertEquals(2, Message.MessageData.returnTotalMessages());
}

//PART 3 - HELPER THAT LOADS THE FIVE OFFICIAL TEST MESSAGES INTO THE ARRAYS//
private void loadPart3TestData() {
    Message.MessageData m1 = new Message.MessageData(1, "+27834557896", "Did you get the
cake?",
"1000000001");
    Message.MessageData m2 = new Message.MessageData(2, "+27838884567", "Where are you?
You are late!
I have asked you to be on time.", "1000000002");

```

```

    Message.MessageData m3 = new Message.MessageData(3, "+27834484567", "Yohoooo, I am
at your gate.",
"1000000003");
    Message.MessageData m4 = new Message.MessageData(4, "0838884567", "It is dinner time !",
"1000000004");
    Message.MessageData m5 = new Message.MessageData(5, "+27838884567", "Ok, I am leaving
without
you.", "1000000005");
    m1.sendMessage(1);
    m2.sendMessage(3);
    m3.sendMessage(2);
    m4.sendMessage(1);
    m5.sendMessage(3);
}
//PART 3 - SENT MESSAGES ARRAY TEST//
@Test
public void testSentMessagesArrayPopulated() {
//SENT MESSAGES ARRAY SHOULD CONTAIN MESSAGE 1 AND MESSAGE 4 TEXT - SHOULD
PASS//
    loadPart3TestData();
    boolean hasCake = false;
    boolean hasDinner = false;
    for (Message.MessageData m : Message.MessageData.sentMessages) {
        if (m.getMessageText().equals("Did you get the cake?")) hasCake = true;
        if (m.getMessageText().equals("It is dinner time !")) hasDinner = true;
    }
    assertTrue(hasCake);
    assertTrue(hasDinner);
}
//PART 3 - LONGEST MESSAGE TEST//
@Test
public void testDisplayLongestMessage() {
//MESSAGE 2 IS THE LONGEST OF ALL FIVE MESSAGES - SHOULD MATCH MESSAGE 2 TEXT//
    loadPart3TestData();
    assertEquals("Where are you? You are late! I have asked you to be on time.",
Message.MessageData.displayLongestMessage());
}
//PART 3 - SEARCH BY MESSAGE ID TEST//
@Test
public void testSearchByMessageID() {
//SEARCHING FOR MESSAGE 4 ID SHOULD RETURN "It is dinner time !" - SHOULD PASS//
    loadPart3TestData();

```

```
assertTrue(Message.MessageData.searchByMessageID("1000000004").contains("It is dinner  
time !"));  
}
```

```

//SEARCH BY RECIPIENT TEST//
@Test
public void testSearchByRecipient() {
//+27838884567 HAS TWO MESSAGES - SHOULD RETURN BOTH//
loadPart3TestData();
String result = Message.MessageData.searchByRecipient("+27838884567");
assertTrue(result.contains("Where are you? You are late! I have asked you to be on time."));
assertTrue(result.contains("Ok, I am leaving without you."));
}

//PART 3 - DELETE BY HASH TEST//
@Test
public void testDeleteMessageByHash() {
//DELETING MESSAGE 2 BY ITS HASH SHOULD RETURN A SUCCESS MESSAGE - SHOULD PASS//
loadPart3TestData();
String hashToDelete = null;
for (Message.MessageData m : Message.MessageData.storedMessages) {
if (m.getMessageText().equals("Where are you? You are late! I have asked you to be on time."))
{
hashToDelete = m.getMessageHash();
break;
}
}
assertNotNull(hashToDelete);
assertTrue(Message.MessageData.deleteMessageByHash(hashToDelete).contains("successfully
deleted"));
}

//PART 3 - DISPLAY REPORT TEST//
@Test
public void testDisplayReport() {
//REPORT SHOULD INCLUDE HASH, RECIPIENT AND MESSAGE FOR ALL SENT MESSAGES -
SHOULD PASS//
loadPart3TestData();
String report = Message.MessageData.displayReport();
assertTrue(report.contains("Did you get the cake?"));
assertTrue(report.contains("It is dinner time !"));
assertTrue(report.contains("Message Hash"));
assertTrue(report.contains("Recipient"));
}
}

```

FINAL MESSAGE.JAVA

```
package com.mycompany.message;

import java.util.Scanner;
import java.util.Random;
import java.util.ArrayList;
import java.io.FileWriter;
import java.io.IOException;

public class Message {

    // LOGIN CLASS
    static class Login {
        private String firstName;
        private String lastName;
        private String username;
        private String password;
        private String cellPhoneNumber;

        public Login(String firstName, String lastName, String username, String password, String
cellPhoneNumber) {
            this.firstName = firstName;
            this.lastName = lastName;
            this.username = username;
            this.password = password;
            this.cellPhoneNumber = cellPhoneNumber;
        }

        public boolean checkUserName() {
            return username.contains("_") && username.length() <= 5;
        }

        public boolean checkPasswordComplexity() {
            if (password.length() < 8) return false;

            boolean hasCapital = false, hasNumber = false, hasSpecial = false;

            for (char c : password.toCharArray()) {
                if (Character.isUpperCase(c)) hasCapital = true;
                else if (Character.isDigit(c)) hasNumber = true;
                else if (!Character.isLetterOrDigit(c)) hasSpecial = true;
            }
        }
    }
}
```

```

    }

    return hasCapital && hasNumber && hasSpecial;
}

public boolean checkCellPhoneNumber() {
    // REGEX SOURCED AND ADAPTED FROM:
    // HTTPS://WWW.BAELDUNG.COM/JAVA-REGEX-VALIDATE-PHONE-NUMBERS
    return cellPhoneNumber.matches("^\\+27\\d{9}$");
}

public boolean loginUser(String enteredUsername, String enteredPassword) {
    return this.username.equals(enteredUsername) &&
this.password.equals(enteredPassword);
}

public String returnLoginStatus(String enteredUsername, String enteredPassword) {
    if (loginUser(enteredUsername, enteredPassword)) {
        return "Welcome " + firstName + " " + lastName + ", it is great to see you again!";
    } else {
        return "Username or password incorrect, please try again.";
    }
}

public String getFirstName() { return firstName; }
public String getLastName() { return lastName; }
}

// MESSAGE CLASS
static class MessageData {

    private String messageId;
    private int messageNumber;
    private String recipient;
    private String messageText;
    private String messageHash;
    private String flag;

    static int totalSent = 0;

    static ArrayList<MessageData> sentMessages = new ArrayList<>();
    static ArrayList<MessageData> storedMessages = new ArrayList<>();

```



```

static ArrayList<MessageData> disregardMessages = new ArrayList<>();
static ArrayList<String> messageHashes = new ArrayList<>();
static ArrayList<String> messageIDs = new ArrayList<>();

public MessageData(int messageNumber, String recipient, String messageText) {
    this.messageNumber = messageNumber;
    this.recipient = recipient;
    this.messageText = messageText;
    this.messageID = generateMessageID();
    this.messageHash = createMessageHash();
}

// CONSTRUCTOR WITH FIXED ID USED FOR PRELOADED TEST DATA
public MessageData(int messageNumber, String recipient, String messageText, String
fixedID) {
    this.messageNumber = messageNumber;
    this.recipient = recipient;
    this.messageText = messageText;
    this.messageID = fixedID;
    this.messageHash = createMessageHash();
}

private String generateMessageID() {
    Random rand = new Random();
    long id = (long)(rand.nextDouble() * 9_000_000_000L) + 1_000_000_000L;
    return String.valueOf(id);
}

public boolean checkMessageID() {
    return messageID.length() <= 10;
}

public String checkRecipientCell() {
    if (recipient.startsWith("+") && recipient.length() <= 13) {
        return "CELL PHONE NUMBER SUCCESSFULLY CAPTURED.";
    } else {
        return "CELL PHONE NUMBER IS INCORRECTLY FORMATTED OR DOES NOT CONTAIN
AN INTERNATIONAL CODE. PLEASE CORRECT THE NUMBER AND TRY AGAIN.";
    }
}

public String createMessageHash() {

```

```

String[] words = messageText.trim().split("\\s+");
String firstWord = words[0];
String lastWord = words[words.length - 1];
String idPrefix = messageID.substring(0, Math.min(2, messageID.length()));
return (idPrefix + ":" + messageNumber + ":" + firstWord + lastWord).toUpperCase();
}

public String checkMessageLength() {
    if (messageText.length() <= 250) {
        return "MESSAGE READY TO SEND.";
    } else {
        int over = messageText.length() - 250;
        return "MESSAGE EXCEEDS 250 CHARACTERS BY " + over + "; PLEASE REDUCE THE
SIZE.";
    }
}

public String sentMessage(int choice) {
    switch (choice) {
        case 1:
            flag = "Sent";
            totalSent++;
            sentMessages.add(this);
            messageHashes.add(messageHash);
            messageIDs.add(messageID);
            return "MESSAGE SUCCESSFULLY SENT.";

        case 2:
            flag = "Disregarded";
            disregardMessages.add(this);
            return "PRESS 0 TO DELETE THE MESSAGE.";

        case 3:
            flag = "Stored";
            storedMessages.add(this);
            messageHashes.add(messageHash);
            messageIDs.add(messageID);
            return "MESSAGE SUCCESSFULLY STORED.";

        default:
            return "INVALID OPTION.";
    }
}

```

```
}
```

```
public static String printMessages() {  
    if (sentMessages.isEmpty()) return "NO MESSAGES SENT.";
```

```
    StringBuilder sb = new StringBuilder();
```

```
    for (MessageData m : sentMessages) {  
        sb.append("MESSAGE ID: ").append(m.messageID)  
          .append(" | HASH: ").append(m.messageHash)  
          .append(" | RECIPIENT: ").append(m.recipient)  
          .append(" | MESSAGE: ").append(m.messageText)  
          .append("\n");  
    }
```

```
    return sb.toString();  
}
```

```
public static int returnTotalMessages() {  
    return totalSent;  
}
```

```
// STORES STORED MESSAGES TO JSON FILE  
// FILE WRITING APPROACH ADAPTED FROM:  
// HTTPS://WWW.BAELDUNG.COM/JAVA-WRITE-TO-FILE
```

```
public static void storeMessage(String filename) {  
    StringBuilder json = new StringBuilder();  
    json.append("[\n");
```

```
    for (int i = 0; i < storedMessages.size(); i++) {  
        MessageData m = storedMessages.get(i);
```

```
        json.append(" {\n");  
        json.append("  \"messageID\": \"").append(m.messageID).append("\",\n");  
        json.append("  \"messageHash\": \"").append(m.messageHash).append("\",\n");  
        json.append("  \"recipient\": \"").append(m.recipient).append("\",\n");  
        json.append("  \"message\": \"").append(m.messageText).append("\",\n");  
        json.append("  \"flag\": \"").append(m.flag).append("\",\n");  
        json.append(" }");
```

```
        if (i < storedMessages.size() - 1) json.append(",");
```

```

        json.append("\n");
    }

    json.append("]");

    try (FileWriter fw = new FileWriter(filename)) {
        fw.write(json.toString());
        System.out.println("MESSAGES SUCCESSFULLY STORED TO " + filename);
    } catch (IOException e) {
        System.out.println("ERROR WRITING JSON FILE: " + e.getMessage());
    }
}

// PRELOAD TEST DATA
public static void preloadTestData() {
    MessageData m1 = new MessageData(1, "+27834557896", "Did you get the cake?",
"1000000001");
    MessageData m2 = new MessageData(2, "+27838884567", "Where are you? You are late!
I have asked you to be on time.", "1000000002");
    MessageData m3 = new MessageData(3, "+27834484567", "Yohoooo, I am at your gate.",
"1000000003");
    MessageData m4 = new MessageData(4, "0838884567", "It is dinner time !",
"1000000004");
    MessageData m5 = new MessageData(5, "+27838884567", "Ok, I am leaving without
you.", "1000000005");

    m1.sendMessage(1);
    m2.sendMessage(3);
    m3.sendMessage(2);
    m4.sendMessage(1);
    m5.sendMessage(3);
}

// DISPLAY STORED MESSAGES INFO
public static String displaySenderAndRecipient() {
    if (storedMessages.isEmpty()) return "NO STORED MESSAGES.";

    StringBuilder sb = new StringBuilder();

    for (MessageData m : storedMessages) {
        sb.append("MESSAGE NUMBER: ").append(m.messageNumber)
            .append(" | RECIPIENT: ").append(m.recipient)

```

```

        .append("\n");
    }

    return sb.toString();
}

// FIND LONGEST MESSAGE
public static String displayLongestMessage() {
    ArrayList<MessageData> all = new ArrayList<>();
    all.addAll(sentMessages);
    all.addAll(storedMessages);
    all.addAll(disregardMessages);

    if (all.isEmpty()) return "NO MESSAGES AVAILABLE.";

    MessageData longest = all.get(0);

    for (MessageData m : all) {
        if (m.messageText.length() > longest.messageText.length()) {
            longest = m;
        }
    }

    return longest.messageText;
}

// SEARCH BY MESSAGE ID
public static String searchByMessageID(String searchID) {
    ArrayList<MessageData> all = new ArrayList<>();
    all.addAll(sentMessages);
    all.addAll(storedMessages);

    for (MessageData m : all) {
        if (m.messageID.equals(searchID)) {
            return "RECIPIENT: " + m.recipient + "\nMESSAGE: " + m.messageText;
        }
    }

    return "MESSAGE ID NOT FOUND.";
}

// SEARCH BY RECIPIENT

```

```

public static String searchByRecipient(String searchRecipient) {
    ArrayList<MessageData> all = new ArrayList<>();
    all.addAll(sentMessages);
    all.addAll(storedMessages);

    StringBuilder sb = new StringBuilder();

    for (MessageData m : all) {
        if (m.recipient.equals(searchRecipient)) {
            sb.append(m.messageText).append("\n");
        }
    }

    if (sb.length() == 0) return "NO MESSAGES FOUND FOR THAT RECIPIENT.";

    return sb.toString();
}

// DELETE BY HASH
public static String deleteMessageByHash(String hash) {
    ArrayList<MessageData> all = new ArrayList<>();
    all.addAll(sentMessages);
    all.addAll(storedMessages);

    for (MessageData m : all) {
        if (m.messageHash.equalsIgnoreCase(hash)) {
            String deletedText = m.messageText;
            sentMessages.remove(m);
            storedMessages.remove(m);
            messageHashes.remove(m.messageHash);
            messageIDs.remove(m.messageID);
            return "MESSAGE \"" + deletedText + "\" SUCCESSFULLY DELETED.";
        }
    }

    return "HASH NOT FOUND. NO MESSAGE DELETED.";
}

// FULL SENT REPORT
public static String displayReport() {
    if (sentMessages.isEmpty()) return "NO SENT MESSAGES TO REPORT.";

```

```

        StringBuilder sb = new StringBuilder();

        for (MessageData m : sentMessages) {
            sb.append("MESSAGE HASH: ").append(m.messageHash)
              .append(" | RECIPIENT: ").append(m.recipient)
              .append(" | MESSAGE: ").append(m.messageText)
              .append("\n");
        }

        return sb.toString();
    }

    public String getMessageID() { return messageID; }
    public String getRecipient() { return recipient; }
    public String getMessageText() { return messageText; }
    public String getMessageHash() { return messageHash; }
    public String getFlag() { return flag; }
}

// MAIN METHOD
public static void main(String[] args) {

    Scanner scanner = new Scanner(System.in);
    Login user = null;

    // WELCOME SCREEN
    boolean welcomed = true;

    while (welcomed) {
        System.out.println("\nWELCOME TO QUICKCHAT");
        System.out.println("1) Register");
        System.out.println("2) Login");
        System.out.println("3) Exit");
        System.out.print("Choose option: ");

        int welcome;

        try {
            welcome = Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("INVALID OPTION.");
            continue;
        }
    }
}

```

```

    }

    switch (welcome) {
    case 1:
        System.out.println("\nREGISTRATION");
        System.out.print("First Name: ");
        String regFirst = scanner.nextLine();
        System.out.print("Last Name: ");
        String regLast = scanner.nextLine();

        String regUsername;

        while (true) {
            System.out.print("Username: ");
            regUsername = scanner.nextLine();

            Login tempU = new Login(regFirst, regLast, regUsername, "", "+270000000000");

            if (tempU.checkUserName()) {
                System.out.println("USERNAME SUCCESSFULLY CAPTURED.");
                break;
            } else {
                System.out.println("USERNAME INVALID (MUST CONTAIN '_' AND BE ≤ 5
CHARACTERS).");
            }
        }

        String regPassword;

        while (true) {
            System.out.print("Password: ");
            regPassword = scanner.nextLine();

            Login tempP = new Login(regFirst, regLast, regUsername, regPassword,
"+270000000000");

            if (tempP.checkPasswordComplexity()) {
                System.out.println("PASSWORD SUCCESSFULLY CAPTURED.");
                break;
            } else {
                System.out.println("PASSWORD INVALID (8+ CHARS, CAPITAL, NUMBER, SPECIAL
CHAR).");
            }
        }
    }
}

```



```
    }  
}
```

```
String regCell;
```

```
while (true) {  
    System.out.print("Cell number (+27): ");  
    regCell = scanner.nextLine();
```

```
    Login tempC = new Login(regFirst, regLast, regUsername, regPassword, regCell);
```

```
    if (tempC.checkCellPhoneNumber()) {  
        System.out.println("CELL NUMBER SUCCESSFULLY CAPTURED.");  
        break;  
    } else {  
        System.out.println("INVALID CELL NUMBER FORMAT (+27 REQUIRED).");  
    }  
}
```

```
user = new Login(regFirst, regLast, regUsername, regPassword, regCell);  
System.out.println("REGISTRATION SUCCESSFUL!");  
break;
```

```
case 2:
```

```
    if (user == null) {  
        System.out.println("PLEASE REGISTER FIRST.");  
        break;  
    }
```

```
System.out.println("\nLOGIN");  
System.out.print("Username: ");  
String enteredUsername = scanner.nextLine();  
System.out.print("Password: ");  
String enteredPassword = scanner.nextLine();
```

```
System.out.println(user.returnLoginStatus(enteredUsername, enteredPassword));
```

```
if (user.loginUser(enteredUsername, enteredPassword)) {  
    welcomed = false;  
}  
break;
```

```

        case 3:
            System.out.println("GOODBYE!");
            scanner.close();
            return;

        default:
            System.out.println("INVALID OPTION.");
    }
}

// PRELOAD TEST DATA
MessageData.preloadTestData();

System.out.print("\nHOW MANY MESSAGES WOULD YOU LIKE TO SEND? ");
int numMessages = 0;

try {
    numMessages = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("INVALID NUMBER. DEFAULTING TO 0.");
}

boolean running = true;

while (running) {

    System.out.println("\nWELCOME TO QUICKCHAT");
    System.out.println("1) Send Messages");
    System.out.println("2) Show Recently Sent Messages");
    System.out.println("3) Stored Messages");
    System.out.println("4) Quit");
    System.out.print("Choose option: ");

    int choice;

    try {
        choice = Integer.parseInt(scanner.nextLine().trim());
    } catch (NumberFormatException e) {
        System.out.println("INVALID OPTION.");
        continue;
    }
}

```

```

switch (choice) {

case 1:
    for (int i = 1; i <= numMessages; i++) {

        System.out.println("\n--- MESSAGE " + i + " OF " + numMessages + " ---");
        System.out.print("Recipient: ");
        String recipient = scanner.nextLine().trim();

        String messageText;

        while (true) {
            System.out.print("Message: ");
            messageText = scanner.nextLine();

            if (messageText.length() <= 250) {
                System.out.println("MESSAGE READY TO SEND.");
                break;
            } else {
                System.out.println("MESSAGE TOO LONG. REDUCE SIZE.");
            }
        }

        MessageData msg = new MessageData(i, recipient, messageText);

        System.out.println(msg.checkRecipientCell());
        System.out.println("MESSAGE ID: " + msg.getMessageID());
        System.out.println("MESSAGE HASH: " + msg.getMessageHash());

        System.out.println("\n1) Send");
        System.out.println("2) Disregard");
        System.out.println("3) Store");
        System.out.print("Choose: ");

        int sendChoice;

        try {
            sendChoice = Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            sendChoice = 2;
        }
    }
}

```

```

        System.out.println(msg.sendMessage(sendChoice));
    }

    System.out.println("\nTOTAL MESSAGES SENT: " +
        MessageData.returnTotalMessages());

    if (!MessageData.storedMessages.isEmpty()) {
        MessageData.storeMessage("stored_messages.json");
    }

    break;

case 2:
    System.out.println(MessageData.printMessages());
    break;

case 3:
    boolean storedMenu = true;

    while (storedMenu) {
        System.out.println("\nSTORED MESSAGES MENU");
        System.out.println("a) Display sender and recipient");
        System.out.println("b) Longest message");
        System.out.println("c) Search by ID");
        System.out.println("d) Search by recipient");
        System.out.println("e) Delete by hash");
        System.out.println("f) Sent report");
        System.out.println("x) Back");

        String sub = scanner.nextLine().toLowerCase();

        switch (sub) {

            case "a":
                System.out.println(MessageData.displaySenderAndRecipient());
                break;

            case "b":
                System.out.println(MessageData.displayLongestMessage());
                break;

            case "c":

```

```

        System.out.print("Enter ID: ");
        System.out.println(MessageData.searchByMessageID(scanner.nextLine()));
        break;

    case "d":
        System.out.print("Enter recipient: ");
        System.out.println(MessageData.searchByRecipient(scanner.nextLine()));
        break;

    case "e":
        System.out.print("Enter hash: ");
        System.out.println(MessageData.deleteMessageByHash(scanner.nextLine()));
        break;

    case "f":
        System.out.println(MessageData.displayReport());
        break;

    case "x":
        storedMenu = false;
        break;

    default:
        System.out.println("INVALID OPTION.");
    }
}
break;

case 4:
    System.out.println("GOODBYE!");
    running = false;
    break;

default:
    System.out.println("INVALID OPTION.");
}
}

scanner.close();
}
}

```


References

<https://www.baeldung.com/java-write-to-file>

<https://www.baeldung.com/java-regex-validate-phone-numbers>

<https://docs.oracle.com/javase/tutorial/>

<https://www.jetbrains.com/help/idea/>

<https://www.w3schools.com/java/>

<https://junit.org/junit5/docs/current/user-guide/>